

Logistic Regression (Binomial) Algorithm Machine Learning in Python

By Johannes Simatupang, MKom, COBIT5 (April 2021)

COMP6577 - Machine Learning - LB01 >>> Bina Nusantara University

Description: Logistic regression is one of the most popular Machine Learning algorithms, which comes under the Supervised Learning technique. Logistic regression predicts the output of a categorical dependent variable. Logistic regression is used for solving the classification problems.

Type of Logistic Regression:

1. Binomial: In binomial Logistic regression, there can be only two possible types of the dependent variables, such as 0 or 1, Pass or Fail, etc.
2. Multinomial: In multinomial Logistic regression, there can be 3 or more possible unordered types of the dependent variable, such as "cat", "dogs", or "sheep"
3. Ordinal: In ordinal Logistic regression, there can be 3 or more possible ordered types of dependent variables, such as "low", "Medium", or "High".

In this Binomial Logistic Regression in Python example, the data set that we will be using is a subset of famous Breast Cancer Wisconsin (Diagnostic) data set. Some of the key points about this data set are mentioned below:

-Four real-valued measures of each cancer cell nucleus are taken into consideration here. * Radius_mean represents mean radius of cell nucleus * Texture_mean represents mean texture of cell nucleus * Perimeter_mean represents mean perimeter of cell nucleus * Area_mean represents mean area of cell nucleus
-Based on these measures the diagnosed result is divided in two categories, malignant and benign. * Diagnosis column consists of two categories, malignant (M) and benign (B)

Step for Logistic Regression :

-1) Data Pre-processing step -2) Fitting Logistic Regression to the Training set -3) Predicting the test result -4) Test accuracy of the result(Creation of Confusion matrix)

In [32]:

```
#1) Data Pre-processing step
# importing libraries
import numpy as nm
import matplotlib.pyplot as mtp
import pandas as pd

#importing datasets
dataset= pd.read_csv('datasets/cancer_data.csv')
```

In [33]:

```
#print dataset  
dataset
```

Out[33]:

	radius_mean	texture_mean	perimeter_mean	area_mean	diagnosis
0	17.99	10.38	122.80	1001.0	M
1	20.57	17.77	132.90	1326.0	M
2	19.69	21.25	130.00	1203.0	M
3	11.42	20.38	77.58	386.1	M
4	20.29	14.34	135.10	1297.0	M
...	...	...	...	...	...
564	21.56	22.39	142.00	1479.0	M
565	20.13	28.25	131.20	1261.0	M
566	16.60	28.08	108.30	858.1	M
567	20.60	29.33	140.10	1265.0	M
568	7.76	24.54	47.92	181.0	B

569 rows × 5 columns

In [38]:

```
#Split the data into features(X) and target(Y) Label sets  
X=pd.DataFrame(dataset.iloc[:, :-1])  
y=pd.DataFrame(dataset.iloc[:, -1])
```

In [35]:

X

Out[35]:

	radius_mean	texture_mean	perimeter_mean	area_mean
0	17.99	10.38	122.80	1001.0
1	20.57	17.77	132.90	1326.0
2	19.69	21.25	130.00	1203.0
3	11.42	20.38	77.58	386.1
4	20.29	14.34	135.10	1297.0
...	...	...	...	...
564	21.56	22.39	142.00	1479.0
565	20.13	28.25	131.20	1261.0
566	16.60	28.08	108.30	858.1
567	20.60	29.33	140.10	1265.0
568	7.76	24.54	47.92	181.0

569 rows × 4 columns

In [39]:

print(y)

```

diagnosis
0      M
1      M
2      M
3      M
4      M
..     ...
564    M
565    M
566    M
567    M
568    B

```

[569 rows x 1 columns]

In [43]:

```

#2) Fitting Logistic Regression to the Training set
#import module to split dataset
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.2, random_state=1)

```

In [44]:

```
#Import module for fitting
from sklearn.linear_model import LogisticRegression
#Create Instance of Logistic Regression
logmodel = LogisticRegression()
print(logmodel.fit(X_train,y_train))
```

C:\ProgramData\Anaconda3\lib\site-packages\sklearn\linear_model\logistic.py:432: FutureWarning: Default solver will be changed to 'lbfgs' in 0.22. Specify a solver to silence this warning.

FutureWarning)

C:\ProgramData\Anaconda3\lib\site-packages\sklearn\utils\validation.py:724: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

y = column_or_1d(y, warn=True)

```
LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,
                    intercept_scaling=1, l1_ratio=None, max_iter=100,
                    multi_class='warn', n_jobs=None, penalty='l2',
                    random_state=None, solver='warn', tol=0.0001, verbose=0,
                    warm_start=False)
```

In [45]:

```
#3) Predicting the test result
y_pred = logmodel.predict(X_test)
```

In [46]:

```
#4) Test accuracy of the result(Creation of Confusion matrix)
from sklearn.metrics import confusion_matrix
confusion_matrix = confusion_matrix(y_test, y_pred)
print(confusion_matrix)
```

```
[[67  5]
 [11 31]]
```

Catatan dari atas: True Positive = 67 True negative = 31 False positive = 5 False negative = 11

source: <https://intellipaat.com/blog/confusion-matrix-python/> (<https://intellipaat.com/blog/confusion-matrix-python/>).